UNIT 6

Q1) Explain perceptron learning algorithm? Describe shortly about how learning parameters are updated in multi-layer perceptron.

The Perceptron: So to make the artificial neurons more intelligent and logical, artificial neurons need to work together as a network of neurons, forming an artificial neural network, mimicking a biological neural network.

McCulloch and Pitts neuron model only considers the inputs, the weights, and the threshold for generating an output. It ignores the connection happening between the neurons in the way that they are interconnected. A neuron needs to learn how you should change the weights and the thresholds of the neurons interconnected together so that the neural network learns and makes the right answer.

Definition: The Perceptron is a computational model invented by Warren McCulloch and Walter Pitts in 1943, inspired by the structure of a biological neuron. It is the most basic form of a neural network

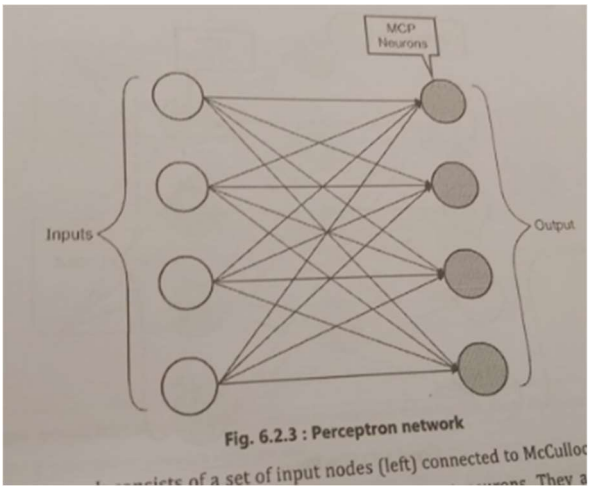


Fig. 6.2.3 : Perceptron network

- The Perceptron network consists of a set of input nodes (left) connected to McCulloch and Pitts neurons (right) using weighted connections. Note here that the inputs are just drawn to make the diagram visually symmetric.
- The neurons in the Perceptron are completely independent of each other. Each neuron independently works to aggregate all the weighted inputs and then use the activation function to decide whether to fire. The neurons might get the same input but with different weights attached to them. So, all the neurons receive all the inputs but could be with different weights.
- Also, note here that unlike the diagram of a Perceptron network shown here, you may have a higher or a smaller number of neurons compared with the number of inputs. For e.g., you could have 4 inputs fed to 3 neurons or 5 inputs fed to 11 neurons. The number of inputs need not be the same as the number of neurons. The number of inputs is determined by the data. Also, the number of outputs is determined by the target value that you wish to get out of supervised learning.
- Mathematically, in general, in a Perceptron network there will be 'n' inputs and 'm' neurons. In McCulloch and Pitts neuron, the weights were labeled as W, where 'i' could range from 1 to m.

In Perceptron network, each neuron can get its own specific weight. So, the weights are denoted as $\mathbf{W}_{ij}$, where index 'i' denotes the input node number and index 'j' represents the specific neuron to which that weight and input needs to be fed. For example, W32 is the weight that connects input node 3 to neuron 2.

Multilayer Perceptron

- Similarly, if a neural network requires complex decision making, you can create multiple layers of perceptron network.
- Each layer takes inputs from the previous layer. That way, the inputs to further layers become refined, adjusted, and you could build a complex decision-making (or classification) system.
- In this multilayer perceptron network, layer 1 processes the first set of inputs. The outputs from various neurons in layer 1 become inputs for layer 2 neurons. Generally, the outputs from the previous layer are provided weights as well as they are fed to the subsequent layers. So, layer 2 would get weighted inputs from layer 1 Similarly, layer 3 would get weighted inputs from layer 2. Finally, layer 3 neuron could produce the desired output.
- Each perceptron makes a decision by weighing up the results from the previous layer of decision-making. In this

way a perceptron in the second layer can make a decision at a more complex and more abstract level [1] than a perceptron in the first layer. Perceptrons at third layer can carry out even more complex decisions. In this way, a many-layer network of perceptrons can engage in sophisticated decision making.

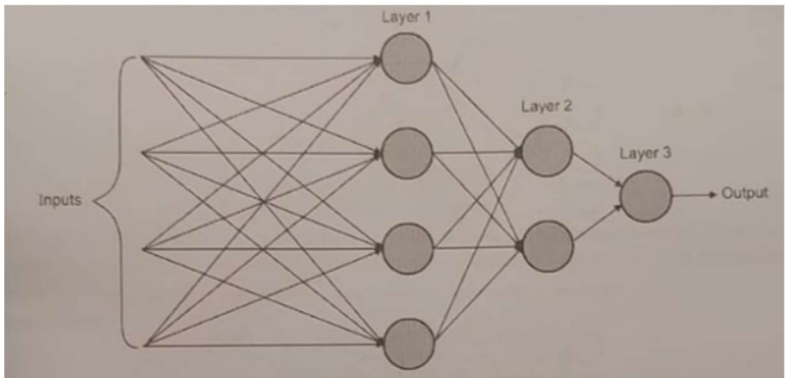


Fig. 6.2.5 : Multilayer Perceptron Network

Bias Input

- Since, each MCP neuron has a threshold value that determines if the neuron fires or not. But what if all the inputs are 0? Even if you have specific weights assigned, those do not matter as all the inputs are 0 and anything multiplied with 0 would give 0. So, the only way you could decide whether a neuron fires or not in such a condition would be by adjusting the threshold. But, adjusting thresholds in various conditions is not practical and feasible. That is where a bias node comes into the picture.
- The bias node is a special node that always has a constant input value of 1. It is connected to each neuron with a weight, and this weight can be adjusted during training. By adjusting the bias weight, we can effectively shift the neuron's activation threshold without changing the weights of the other inputs. This allows the neuron to fire even when all the other inputs are 0.
- So, the bias node adds a degree of flexibility to the neuron's decision-making process. It allows the neuron to learn more complex patterns and relationships in the data.

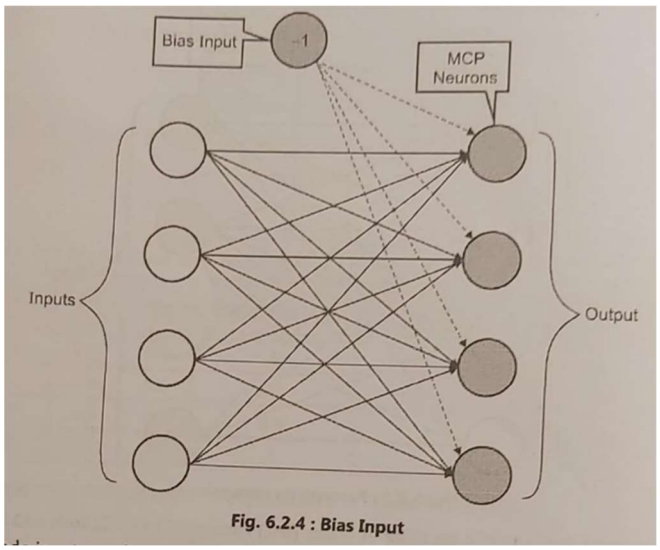


Fig. 6.2.4 : Bias Input

 A biased input node is set to a fixed random arbitrary value such that it is non-zero. It could be either positive or negative. But, an input of -1 is commonly used as a biased input. So, now, you can adjust the weights of this biased input so as to fire one or more neurons as desired even if other legitimate inputs are all 0. This allows you to use the same algorithm without requiring new parameters in the equation.

Q2) **Activation Functions , Types of Activation Functions**

Definition: Activation function decides whether the artificial neuron would fire or not for a given set of inputs.

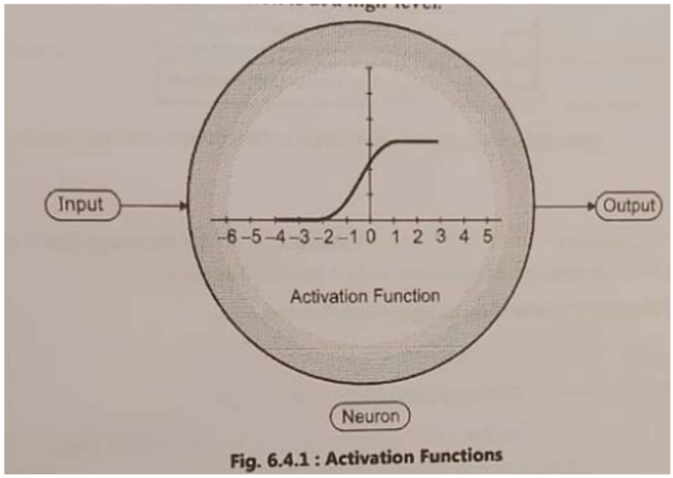


Fig. 6.4.1 : Activation Functions

- The activation function is a mathematical "gate" in between the input feeding the current neuron and its output going to the next layer. It can be as simple as a step function that turns the neuron output on and off , depending on a rule or threshold.
- It can also be a transformation that maps the input signals into output signals that are needed for the neural network to function.
- Activation function is a crucial component of any neural network. It determines the output of a neural network model, its accuracy, and also the computational efficiency of training a model.
- Activation functions also play a major role in deciding the speed at which a neural network can generate output when provided with inputs.
- Activation functions are mathematical equations that work on provided input values. Each neuron in the network has a corresponding activation function. Different neuron layers may have different activation functions based on their roles and the desired behaviour.
- It is also desired that activation functions must be computationally efficient because they are calculated across thousands or even millions of neurons for each data sample

Types of Activation Functions

1. Identity Function 2. Binary Step Function
3. Sigmoid/Logistic Function 4. Tanh/Hyperbolic Tangent Function
5. ReLU (Rectified Linear Unit) Function

1. Identity Function

The simplest activation function is the identity function that passes on the incoming signal as the outgoing signal without any change or processing. It just replicates what is provided as input.

Mathematically, the identity activation function g(x) is defined as: **g(x) = x**

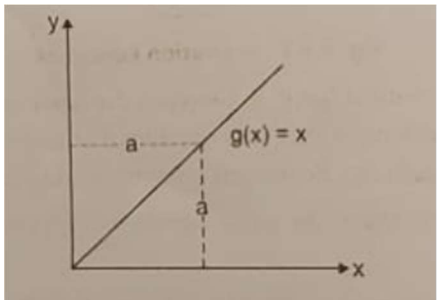


Fig. 6.4.3 : Identity Function Graph

You typically use the identity function when you just want to forward the weighted input (input multiplied by the respective weight) to the next layer of neurons without any additional processing.

2. Binary Step Function

- A binary step function is a threshold-based activation function. If the input value is above a certain threshold, the neuron is activated, and it produces an output of 1. If the input value is below a certain threshold, the neuron is not activated, and it produces an output of 0. This is the activation function we have used so far in our discussions

- Mathematically, the binary step activation function g(x) is defined as: **g(x) = 1 when x > 0, otherwise 0**
The Fig. 6.4.4(a) shows the binary step function graphically.

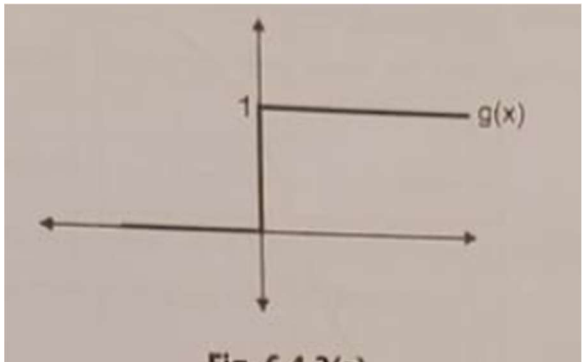


Fig. 6.4.3(a)

If you have a specific threshold value, then the step function is mathematically represented as:
g(x) = 1 when x > threshold, otherwise 0
The Fig. 6.4.3(b) shows the binary step function graphically.

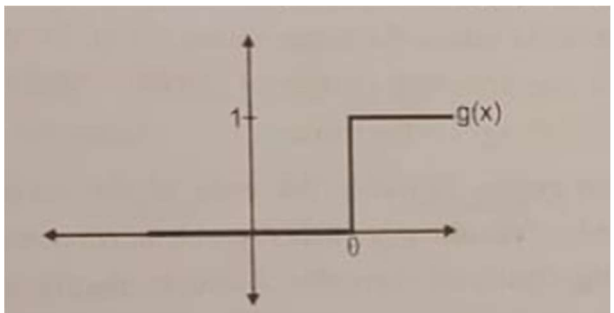


Fig. 6.4.3(b) : Binary Step Function

- One common problem with a binary step function is that it does not produce different valued outputs (such as 0.1, 0.2, 0.5, etc.). Hence, it cannot be used for classifying the inputs into one of the several categories. It can only produce two outputs, 0 or 1, and hence can only classify an input into one output category.
- Another problem with step function is that it is not continuous and hence cannot be used to compute mathematical differentiation.

3. Sigmoid/Logistic Function

Sigmoid function takes an input and may produce any output between 0 and 1. It is easy to work with and has all the desired properties of an activation function such as the following:

- It is non-linear
- It is continuously differentiable
- It is monotonic
- It has a fixed output range

Sigmoid functions are one of the most widely used activation functions today.

Mathematically, sigmoid function S(x) is defined as: $S(x) = \frac{1}{1+e^{-x}}$

The Fig. 6.4.4 shows sigmoid function graphically. It is also called as S-curve.

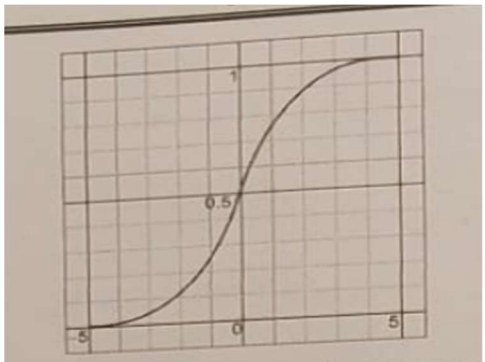


Fig. 6.4.4 : Sigmoid / Logistic Function

- As you notice from the plot, sigmoid function is non-linear. Hence, unlike step functions, it can give a range of values. Also, any small change in the values of x causes values of y to change significantly. So, it can also work as a step function where required. One drawback of sigmoid function is that towards the end of the curve, y values change very less as compared to changes in the x value.
- The gradient (slope) at that region (towards the ends of the curve) becomes increasingly smaller and insignificant. This is also called as "vanishing gradients" problem.When you reach this stage , the neural network model does not learn anything significant thereafter. However,despite this Sigmoid Function is most commonly used activation function.

4. TanH/Hyperbolic Tangent Function

- TanH or Hyperbolic Tangent is another commonly used activation function. It is very similar to sigmoid function.
- It takes an input and may produce any output between -1 and 1. It is easy to work with and has all the desired properties of an activation function such as the following:

-It is non-linear
- It is continuously differentiable
- It is monotonic
- It has a fixed output range
- Its output is zero-centered (making it easier to model inputs that have strongly negative, neutral, and strongly positive values)

Mathematically, TanH function is defined as: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

The Fig. 6.4.5 shows TanH function graphically. As you notice, it looks very similar to sigmoid curve

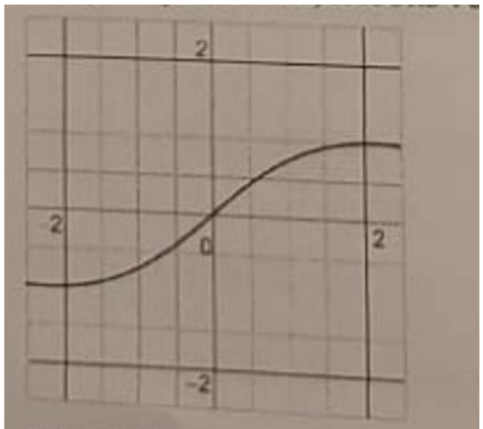


Fig. 6.4.5 : TanH Function

- Similar to sigmoid function, TanH function also has "vanishing gradient" problem where towards the end of the TanH curve, y values change very less as compared to changes in the x value.
- The gradient (slope) at that region (towards the ends of the curve) becomes increasingly smaller and insignificant.

5. ReLU (Rectified Linear Unit) Function

- ReLU is one of the latest activation functions. It is computationally more efficient and allows the neural network to respond very quickly. It also has the following characteristics:

-It is non-linear
- It is continuously differentiable (except at 0)
- It supports backpropagation
- It does not have a fixed output range
-It is not zero-centered
- It does not have vanishing gradient problem

Mathematically, ReLU function is defined as:

$$R(x) = \begin{cases} 0 \text{ for } x \le 0 \\ x \text{ for } x > 0 \end{cases} = \max(0, x)$$

The Fig. 6.4.6 shows ReLU function graphically.

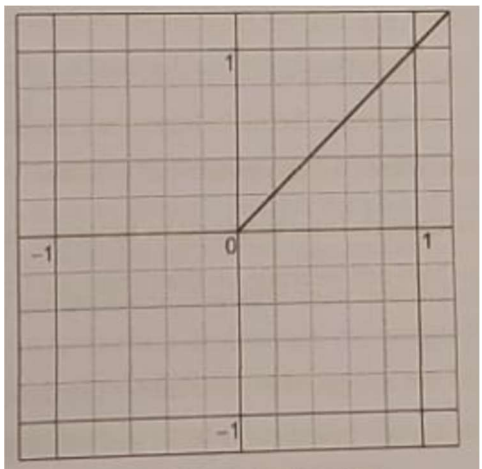


Fig. 6.4.6 : ReLU Function

- One drawback of ReLU function is that when inputs approach zero, or are negative, the gradient of the function becomes zero.
- Consequently, the neural network cannot perform backpropagation and cannot learn further. It is called Dying ReLU problem.

Q3) What is ANN? Explain McCulloch Pitts Neuron.

Artificial Neural Networks (ANN)

Definition: Artificial Neural Networks (ANNs) are inspired by biological neural networks. ANNs try to mimic the computational structure (not biological structure) and working of biological neurons to establish learning and response modes that would work on various inputs and produce the appropriate output.

Structure: ANNs are made up of a number of simple, highly interconnected processing elements, which process information by their dynamic state response to external input.

So, if these functions are diagrammatically represented (without involving biological structures or complexities) to outline the key aspects of how neurons work, then it could possibly look something like as shown in Fig. 6.2.1.

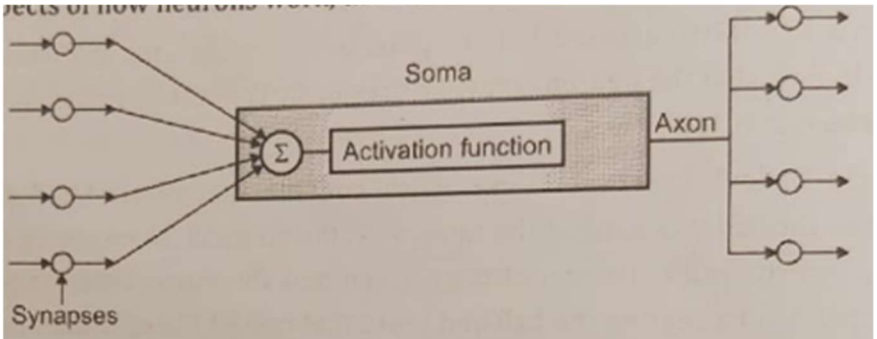


Fig. 6.2.1 : Artificial Neural Networks

This forms the basis of building an ANN, taking an inspiration from biological neuron. An ANN is a group of interconnected artificial neurons, interacting with one another in a collaborative manner.

In such a system, excitation (input signals) is applied to the input of the network and the input is multiplied by appropriate weights as the input signal passes through the network. Based on the accumulated weighted signals, the activation function decides if the artificial neuron would produce output (get activated) or not.

McCulloch-Pitts (MCP) Neuron Model

- In 1943 (yes, 1943), Warren McCulloch and Walter Pitts proposed a mathematical model that could represent how neurons work without getting into their biological details and complexities.
- The mathematical model extracts only the bare essentials required to accurately represent a neuron and removing all of the other unnecessary details. It is one of the several models used today for modelling an artificial neuron.
- The model is represented as shown in Fig. 6.2.2.

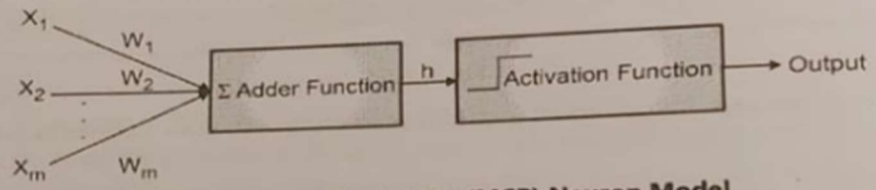


Fig. 6.2.2 : McCulloch-Pitts (MCP) Neuron Model

- $X_1 \ldots X_m$ are inputs.
- $W_1 \ldots W_m$ are respective weights for those inputs. Note here that weights could be positive or negative. Positive weights are called excitatory and they make it more likely that the neuron would fire. Negative weights are called inhibitory and they make it less likely that the neuron would fire.
- Adder function sums (accumulates) the weighted inputs (inputs multiplied by their respective weights) and gives out *h* as the output
- Activation function (threshold function) decides whether the artificial neuron fires ('spikes' or produces output) given the current sum of weighted inputs. When a neuron fires it gives output as 1, else it gives an output of 0.
- So, mathematically, it could be written as

$$h = \sum_{i=1}^{m} W_i X_i$$

Suppose the threshold above which the neuron fires is denoted by θ. Then,
If h>θ, then the neuron fires and output is 1.
If h≤θ, then the neuron does not fire, and output is 0.

Q4) What is vanishing gradient descent problem.= **Sigmoid/Logistic Function**

Q5) With the help of suitable diagram explain Biological Neuron.

Cells within the nervous system called neurons communicate with each other in unique ways. The neuron is the basic working unit of the brain. [1] It is a specialized cell designed to transmit information to other nerve, muscle, or gland cells. It is often cited that our brain has approximately 100 billion interconnected neurons!

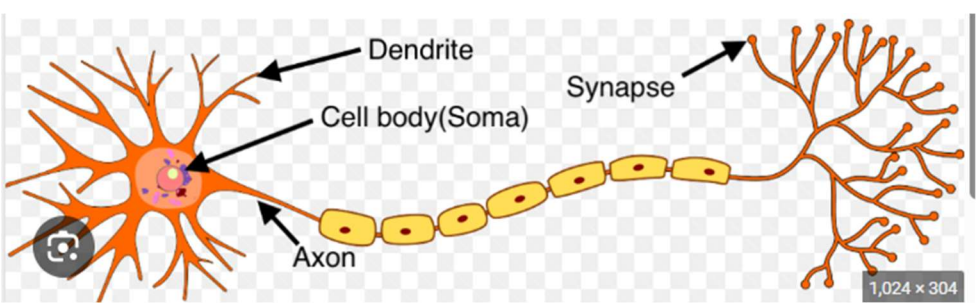


Fig. shows a high-level outline of a neuron.

1. Soma (Cell Body): Soma is the core of a neuron. It is also called as cell body. It carries genetic information, maintains the neuron's structure, and provides energy to drive activities. Like other cell bodies, a neuron's soma contains a nucleus and specialized organelles. It is enclosed by a membrane which both protects it and allows it to interact with its immediate surroundings. Treat soma as the neuron's processor. The dendrites collect stimuli from the neighbouring neurons and pass it on to soma. The soma accumulates the stimuli received through the dendrites. It 'fires' when sufficient stimuli are obtained.

2. **Axon:** An axon is a long, tail-like structure which joins the cell body at a specialised junction called the axon hillock. Many axons are insulated with a fatty substance called myelin. Myelin helps axons to conduct an electrical

signal. Neurons generally have one main axon. You can treat axon as the output device of a neuron. Axon connects neurons together to form a large electrical circuit.
3. **Dendrites:** Dendrites are fibrous roots that branch out from the cell body. Like antennae, dendrites receive and process signals from the axons of other neurons. Neurons can have more than one set of dendrites, known as dendritic trees. You can treat dendrites as input devices to a neuron.
4. **Synapses:** Neurons send signals using action potentials. An action potential is a shift in the neuron's electric potential caused by the flow of ions in and out of the neural membrane. Signals pass from an axon to a dendrite through a junction called synapses. Synapses can connect an axon to another axon or a dendrite to another dendrite. Action potentials can trigger both chemical and electrical synapses.
- One thing to note here is that there is a small gap between the end of an axon and the adjacent dendrite of the neighbouring neuron. This gap is called a synapse. A nervous stimulus is an electric impulse. It is transmitted across a synaptic gap by means of electrochemical processes. The synaptic gap has an important role to play in the activities of the nervous system.

Q6) Explain what is Deep Learning and its different architectures? State the various applications of deep learning.

Deep Learning is a subset of **machine learning** that mimics the functioning of the human brain in processing data and creating patterns for decision-making. It uses **artificial neural networks** with multiple layers (hence "deep") to model complex patterns in data. Each layer learns to extract specific features, with deeper layers learning more abstract features.

Deep Learning is particularly effective for tasks involving large datasets and complex problems, such as image recognition, natural language processing, and speech recognition.

Key Characteristics of Deep Learning:

1. **Multiple Layers**: Deep architectures with multiple layers extract higher-level features from raw input.
2. **Automatic Feature Extraction**: No manual feature engineering; the model learns relevant features on its own.
3. **Scalability**: Performs well on massive datasets due to computational advancements (e.g., GPUs).
4. **End-to-End Learning**: Directly maps input data to output predictions.

Different Architectures of Deep Learning

1. **Feedforward Neural Networks (FNNs)**:
The simplest type of artificial neural network.
Information flows in one direction, from input to output, without cycles.
Used for tasks like classification and regression.
2. **Convolutional Neural Networks (CNNs)**:
Specialized for processing structured grid data, such as images.
Incorporates convolutional layers for feature extraction and pooling layers for dimensionality reduction.
Applications: Image classification, object detection, medical imaging.
3. **Recurrent Neural Networks (RNNs)**:
Designed for sequential data where past information influences future predictions.
Includes feedback connections to process temporal information.
Variants: Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU).
Applications: Language modeling, time-series prediction, speech recognition.
4. **Transformers**:
Revolutionized natural language processing (NLP).
Processes data in parallel using attention mechanisms.
Applications: Language translation, chatbots, and large-scale language models (e.g., GPT, BERT).
5. **Deep Belief Networks (DBNs)**:
Composed of stacked layers of restricted Boltzmann machines (RBMs).
Effective for feature extraction in unsupervised learning.

Applications of Deep Learning

1. Computer Vision:

Image recognition (e.g., facial recognition).

Object detection (e.g., autonomous vehicles).

Medical imaging (e.g., tumor detection).

2. **Natural Language Processing (NLP)**:

Sentiment analysis.

Machine translation.

Chatbots and virtual assistants.

3. **Speech Recognition**:

Voice assistants (e.g., Siri, Alexa).

Transcription services.

4. **Healthcare**:

Disease prediction (e.g., cancer detection).

Drug discovery and genomics.

5. **Gaming**: AI agents for strategy-based games (e.g., AlphaGo).

6. **Finance**:

Fraud detection.

Algorithmic trading.

7. **Robotics**:

Motion control and navigation.

Object manipulation.

8. Recommendation Systems:Personalized content recommendation (e.g., Netflix, Amazon).

Q7) Explain why we use non-linearity function? State and explain three types of neurons that add non-linearity in their computations.

Why Do We Use Non-Linearity in Neural Networks?

Non-linearity is crucial in neural networks because it enables them to learn and model complex patterns in data. Without non-linear activation functions, a neural network with multiple layers would behave like a single-layer network, as linear operations cannot capture intricate relationships in the data. Non-linear functions introduce the ability to handle **non-linear decision boundaries** and solve real-world problems such as image recognition and natural language processing.

Advantages of Using Non-Linear Functions

1. **Captures Complex Patterns**: Enables the model to learn relationships beyond linear correlations.

2. **Enhances Model Capacity**: Allows the network to approximate any continuous function (as per the Universal Approximation Theorem).

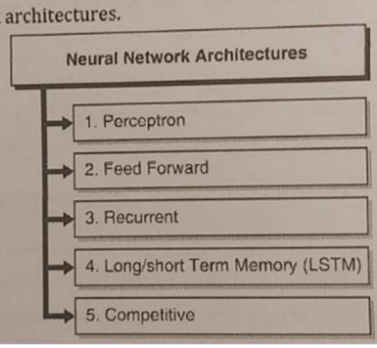
3. **Enables Stacking Layers**: Adds depth to the network, making it possible to build hierarchical feature representations.

three types of neurons that add non-linearity:

1. **Sigmoid Neuron**:
   - **Definition**: Uses the sigmoid function $f(x) = \frac{1}{1+e^{-x}}$, which maps any input to a range between 0 and 1.
   - **Properties**:
     - Smooth and differentiable.
     - Commonly used for binary classification.
   - **Limitations**:
     - Can suffer from vanishing gradient problems.
     - Output saturates for large or small inputs, reducing gradient updates.
   - **Use Case**: Binary classification tasks.
2. **ReLU (Rectified Linear Unit) Neuron**:
   - **Definition**: Uses the ReLU function $f(x) = \max(0, x)$, which outputs the input if it's positive and 0 otherwise.
   - **Properties**:
     - Simple and computationally efficient.
     - Solves the vanishing gradient problem by maintaining gradients for positive inputs.
   - **Limitations**:
     - Can suffer from the "dying ReLU" problem, where neurons become inactive for negative inputs.
   - **Use Case**: Widely used in deep learning models, especially CNNs.
3. **Tanh (Hyperbolic Tangent) Neuron**:
   - **Definition**: Uses the tanh function $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$, which maps inputs to a range between -1 and 1.
   - **Properties**:
     - Similar to sigmoid but centers outputs around 0.
     - Helps with gradient propagation compared to sigmoid.
   - **Limitations**:
     - Also suffers from vanishing gradient issues for large input values.
   - **Use Case**: Suitable when inputs are centered around zero, improving optimization.

Q8) **Neural Network Architecture**

There are several ways in which a neural network could be created. These ways are referred as neural network architecture. You choose a neural network architecture as appropriate for the problem in hand. Let's learn about some of the common neural network architectures.



1. **Perceptron**

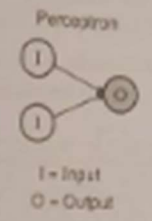


Fig. 6.3.2 : Perceptron

You have already learnt about the perceptron. It is the simplest possible neural network. It can have various inputs but does not have any hidden layers between the input and the output. It is commonly used for classification.

2. **Feed Forward**

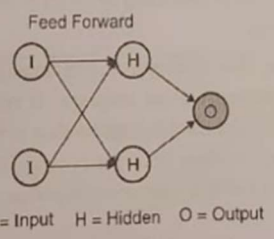


Fig. 6.3.3 : Feed Forward

- In a feed-forward neural network, all the perceptrons are arranged in layers. The hidden layers are not connected with the outside world. There could be one or more hidden layers (so a feed forward network could be shallow or deep as you learnt earlier). Also, every perceptron in one layer is connected with each node in the next layer. Therefore, all the nodes are fully connected.
- In a feed-forward neural network, neurons in the same layer are not interconnected. Hence, the neurons in the same layer work independently of each other. In such a network, algorithms such as backpropagation are commonly used to minimise errors when choosing the weights. Such a neural network is commonly used for pattern recognition, speech recognition, and character recognition.

3. **Recurrent Neural Network**

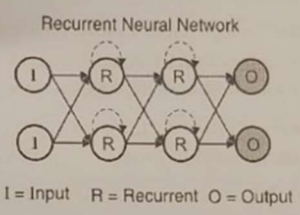


Fig. 6.3.4 : Recurrent Neural Network

- Recurrent neural networks (RNNs) are a variation to feed-forward networks. In this type, each neuron in hidden layers receives an input with a specific delay in time. This type of neural network is used when there is a need to access previous information in current iteration.
- For example, when you are trying to predict the next word in a sentence, you need to know the previously used words first. The neural network computations consider the prior information fed to the neurons.
- However, it is slower when compared with the regular feed forward neural network. It is commonly used for language translation, speech to text conversion, and robotic control.

3. **Long / Short Term Memory (LSTM) Neural Network**

3. Long / Short Term Memory (LSTM) Neural Network

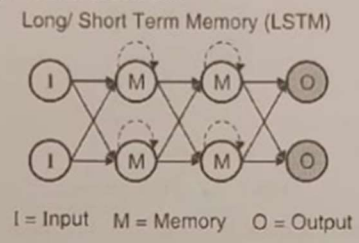


Fig. 6.3.5 : Long / Short Term Memory (LSTM) Neural Network

- LSTM networks treat neurons as a memory cell. They can process data with larger memory gaps. RNNs cannot remember data from a long time ago but LSTMs can. For example, an RNN can predict the next word from what has been written just a few seconds ago.
- But, if you were to predict language spoken in a country, you would need to "remember" which all languages are spoken in that country and which amongst those languages is relevant in the given prediction context. For example, consider that you need to predict the language spoken in the sentence "I grew up in Tamil Nadu. I am fluent in _____". What do you predict? Possibly, a set of languages spoken around that state such as Tamil, Kannada, Malayalam, etc. So, you need LSTM in such a scenario that "remembers" the context for a longer period of time.

4. Competitive Neural Network

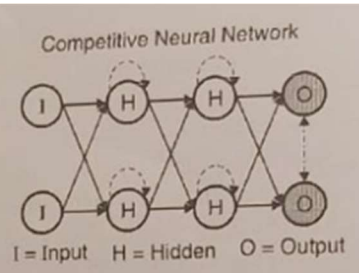


Fig. 6.3.6 : Competitive Neural Network

- Competitive neural networks are structurally similar to feed forward neural networks except that the output neurons are connected amongst themselves usually through negative weights. For a given input pattern, the output neurons of a competitive neural network tend to compete among themselves for that input.
- The output neuron that has the highest activation for the given input pattern is declared the winner. If a neuron wins the competition, its weight is updated according to the competitive learning rule.
- The output neurons compete for the right to respond to a subset of the input data. Competitive learning works by increasing the specialisation of each neuron in the network. Individual neurons of the network learn to specialise on groups of similar patterns and become 'feature detectors' for different classes of input patterns.
- Competitive neural network is commonly used in unsupervised learning for clustering data. So, suppose that the data you want to cluster has got 3 features using which you can decide which cluster the particular data point belongs to. In such a scenario, each neuron could specialise in detecting one of the three features. So, if that particular neuron fires, then it means that the particular feature is present.